金融投研智能体

可验证 · 可追溯 · 可复现的投研 AI 系统

2026 年北京市大学生金融人工智能竞赛 · 参赛作品

选题 1 数据结构化提取 · 选题 4 自动化估值建模 · 选题 5 研究报告纠错核查

源文件 项目计划书.md —— 本 PDF 由 tools/make_plan_pdf.py 生成，可重新生成

一、项目概述

1.1 一句话

我们用自研的 Agent Harness（零第三方编排框架）把「原始投研材料 → 可信事实库 → 估值模型 → 报告核查」串成一条闭环流水线，其中每一个数字都能回溯到原文的哪一页、
哪一行、哪一个表格单元格，每一次运行都能逐字节复现。

1.2 我们要解决的问题

金融投研场景里，AI 最容易出的问题不是"不会算"，而是看起来算对了：

现象	典型后果
模型读出 583,965.66 时把折行的 583,96□5.66 读成 58,396	错一位数量级，报告里毫无痕迹
行标签写着「单位：万元」而正文按元处理	现金放大 100 倍，DCF 每股价值从 ¥12 变成 ¥124
用净利润冒充 EBIT	自由现金流被系统性高估，估值偏高却"逻辑自洽"
报告引用的数字在原始材料里根本不存在	幻觉入文，读者无从核对

这些错误的共同点是：程序不报错、阶段状态是 success、报告看起来很正常。
本项目的全部设计都围绕一件事：让这类错误无法静默通过。

1.3 我们的答案（三条硬机制）

1. 无锚点不入库：任何一条数字必须有原文锚点（页码 / 行号 / bbox）+ 引文回读校验，
否则不进事实库。

2. LLM 只负责"提出"和"解释"，代码负责"确定"和"证明"：模型读数必须由
*它无法影响的*几何/算术事实背书，才允许改写程序读数。

3. 过程全量留痕、产物逐字节可复现：trace.jsonl 逐条记录文件访问、工具调用、
模型调用、断言、逐页路由与阶段；replay --strict 离线重跑并逐字节比对。

二、赛题理解与选题说明

2.1 赛题要求我们做什么

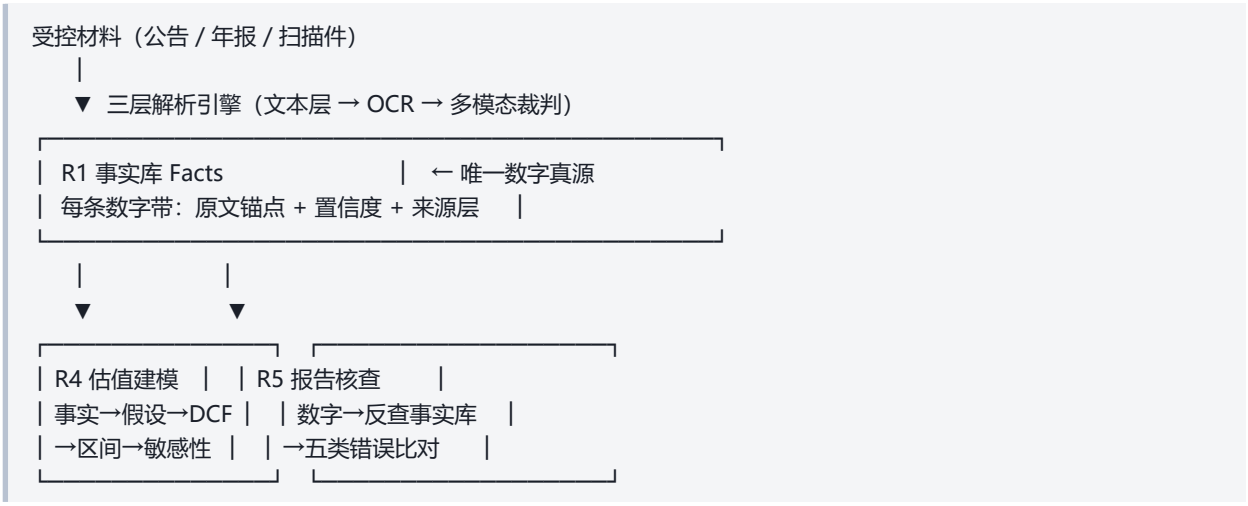
竞赛以「金融投研智能体构建」为主题，要求在受控数据环境中完成
信息检索、数据处理、逻辑核验、量化计算和报告生成，重点考察
金融数据的准确处理能力、复杂任务的组织执行能力、
分析结论的准确性和专业性，以及运行过程的可验证、可追溯和可复现程度。

评分维度共七项：任务完成度、数据与计算准确性、分析逻辑严密性、
证据支撑充分性、结果稳定性、方案创新性、实际应用价值，并结合现场展示与答辩。

2.2 我们选择：选题 1 / 4 / 5

选题	内容	重点考察
选题 1	数据结构化提取	从非标公告、扫描文件、复杂表格中提取质押/中标/股权变动字段；字段完整性、格式规范性、表格解析准确性、结果一致性
选题 4	自动化估值建模	依据财务数据、经营假设与可比公司，建立 DCF 或相对估值模型，给出估值区间与敏感性分析
选题 5	研究报告纠错核查	核查草稿中的收入、利润、估值倍数、单位、统计口径与引用，识别错误并提出修改意见；准确率、覆盖率、误报控制、来源定位

2.3 为什么是这三个：它们不是三个功能，而是一条质量链



关键设计：R4 与 R5 只消费事实库，不重复解析文档。
三个选题共用同一个数字真源，因此「事实库准不准」是一个可以被单独检验的问题 ——
这比做三个互不相干的功能更能体现任务的组织执行能力。

三、系统方案

3.1 总体架构

```
agent/                  ★ 自研 Harness —— 完全领域无关，可驱动任何任务
├── runtime/    loop.py      两级调度：宏观 Plan DAG + 微观有界 Agent Loop
│               budget.py    四道预算闸门（步数/token/时长/成本）
├── model/      gateway.py   模型网关：多角色、缓存、记账、结构化输出自我修复
│               cache.py     响应缓存 —— replay 与成本控制的基础
│               providers/   openai_compat（DeepSeek/Qwen/GLM/vLLM）+ mock
├── tools/      registry.py  工具注册表（副作用/确定性/成本声明）
│               fsguard.py   文件系统守卫（受控数据环境的执行者）
├── prompts/    registry.py  提示词注册表（版本 + SHA256 指纹）
├── skills/     registry.py  技能注册表（加载 SKILL.md 并校验绑定真实存在）
├── mcp/        client.py    MCP 协议客户端（JSON-RPC 2.0 over stdio，自研）
├── validate/   五道校验闸门（单位归一化 / quote 回读 / 勾稽断言 / 契约 / 数值）
├── evidence/   models.py ★ 全局数据契约（单一真源）  store.py 证据库 + 事实账本
└── trace/      tracer.py 全量事件追踪（JSONL）  replay.py 复现校验

domain/                 业务知识（提示词 / 技能 / 财务与文档工具）
pipelines/              r1_extract · r4_valuation · r5_audit · full
web/                    演示台（FastAPI + Jinja2 服务端渲染，无构建链、无 CDN）
```

一条命令即可复现全流程，这是评审进入本作品的方式：

```
python tools/reproduce_all.py    # 12 步，全部零依赖（不需 API Key / 不需联网）
```

最简使用方式（Web 一键）：把要分析的 PDF 放进 inbox/raw/， 浏览器打开 /analyze，勾选费用确认后点「开始分析」—— 系统用真实模型跑完整的 R1→R4→R5，页面上实时显示 11 个阶段的进度与运行日志， 跑完后结果就在 /library /facts /valuation /audit 等页面里 （与命令行 agent run full 的产物完全同源）。

另有控制台 python -m web.panel --run <run_id>（一页四标签：运行 / R1 / R4 / R5，切标签不跳页）。它的右侧常驻一栏模型与工具的原始记录：模型收到的完整提示词、它答的原文、工具收到的入参与返回的原文，一条一条往下流，运行中每 2 秒续读。 这一栏读的是 agent_log.jsonl，与 trace.jsonl（按契约只记载荷哈希）是两路数据 —— 看过程要的是前者。--mock 的离线自检 run 在这一栏里与真实 run 无从分辨，所以页眉会挂出「离线自检」，运行中的 run 则从 checkpoint.json 判读。

3.2 R1 数据结构化提取：三层解析引擎 + 逐页确定性路由

第一层 文本层直取：有文本层的页面直接取字，零成本、零幻觉。

第二层 OCR：无文本层的扫描件走本机 OCR（纯 CPU，无需 GPU）， 配合扫描件预处理链（灰度 → 分辨率归一 → 纠偏 → 去噪）。

第三层 多模态裁判：关键字段交由视觉模型裁决，但模型不能单独决定结果 ——
它的每一次改写都必须由它无法影响的几何事实背书：

· 表格先按单元格 x 区间重叠重建列结构（不看列号，改看坐标）， 实测 730 / 987 张真实表格的列结构被重建；

· 单元格裁决时，程序读数与模型读数并列，只有当模型读数与网格几何（折行位置、数字连续性、量纲）一致时才采纳，且必须通过 quote 回读；

· 没有原文锚点的数值一律不入库。

逐页路由是这套设计的关键：每一页走哪条通道是确定性判据算出来的（字符数、乱码率、文本覆盖率、线段密度），并逐页写入 trace.jsonl ——
评审可以逐页追问「为什么这一页走 OCR」。

3.3 R4 自动化估值建模：事实 → 假设 → 模型 → 区间 → 敏感性

· DCF（FCFF）：显式预测期 + 永续增长；折现率由 CAPM 逐项推导 （无风险利率取自带来源 URL 与日期的市场利率缓存）。

· 相对估值：可比公司倍数法，真实同业快照（含抓取时间戳与来源）， 有意剔除目标公司自身。

· 估值区间：取两个方法结果的并集极值（而非平均）—— 平均会掩盖模型分歧。

· 敏感性：双因素网格热力图 + 单因素龙卷风图 + 区间图。

· 勾稽断言：A1–A6 六类会计恒等式独立重算（不是把模型的中间量拿来比对，那叫自证不叫校验），任一 FAIL 即阻断报告输出。

假设的三种属性在报告里显式标注：绿色代表事实，齿轮图标代表一致性调整，蓝色代表观点。
每条假设可回溯到具体页、具体表格单元格。

3.4 R5 研究报告纠错核查：五类错误 + 零误报优先

类别	检查内容
E1 数值	草稿中的数字能否在事实库中找到来源（无来源即报错）
E2 单位	量纲族隔离 + 单位归一化（万元/亿元/元）
E3 口径	合并/母公司、归母/扣非/全部净利润、时点/期间
E4 引用	引文是否真实存在于原始材料
E5 倍数	静态/TTM/前瞻口径隔离，防止倍数张冠李戴

误报控制是硬约束：完全正确的草稿必须零发现 （test_no_false_positives_on_clean_draft 守着这条线）； 评测黄金集里专门放入 11 条"陷阱项"（看起来像错、其实是对的），用来防止"宁可错杀"的规则漂移。每条发现都给出原文锚点 + 修订值 + 批注版草稿。

3.5 可验证 / 可追溯 / 可复现是怎么做到的

赛题要求	实现
数据来源可核验	事实库每条数字带原文锚点（页/行/bbox）+ 引文 + 置信度 + 来源层
执行过程可追溯	trace.jsonl 记录文件访问、工具调用、模型调用、断言、逐页路由、阶段与错误；agent audit 生成人类可读版
运行结果可复现	响应缓存按内容哈希；thinking 关闭（官方文档确认思考模式下 temperature 不生效）；replay --strict 离线逐字节比对
受控数据环境	默认禁止出网；--offline 下缓存未命中直接报错，不会悄悄联网。读写分离由运行时闸门强制（FileGuard 挂在 RunContext.guard() 上，越界抛 SandboxViolation）；产物侧另强制禁止绝对路径

四、关键创新点

4.1 自研 Agent Harness：零第三方编排框架

评测框架本身不是黑盒 —— 编排、工具注册、提示词版本、技能绑定、MCP 客户端、
追踪与复现全部自研，因此"过程可追溯"不是某个框架的日志开关，
而是这套系统的结构性属性。赛题点名的核心模块（编排框架 / Tool / Prompt / Skill / MCP /
数据处理 / 日志）全部可直接点开查看，并有体检守住文档与代码一致：

```
python -m agent.cli tools          # 17 个工具 / 7 个模块（含副作用与确定性声明）
python -m agent.cli prompts        # 5 个提示词（版本 + SHA256 指纹）
python -m agent.cli skills         # 3 个技能（校验绑定的工具/提示词真实存在）
python -m agent.cli mcp --self-test  # MCP：真实子进程，握手/发现/调用/错误路径
```

4.2 几何佐证驱动的单元格级视觉裁决

模型改写程序读数时，必须由模型无法影响的几何事实背书
（折行位置、数字连续性、量纲一致性、网格对齐），并且裁图作为物证随产物落盘 —— 评审点开
pages/cells/<doc_id>/ 就能看到"模型当时看到的那张图"。

> 没有那张图，"视觉裁决"就只是一句自述。

4.3 三态勾稽断言：未校验 ≠ 失败

A1–A6 断言有三种结果：通过（独立重算一致）、失败（校验过且不一致）、
未校验（缺辅助明细，本次无法独立重算）。三者在报告里分开计数 ——
把"未校验"算成"失败"会虚报问题，把"失败"洗成"未校验"会掩盖问题。
（这条区分由一个正例、一个反例两条测试钉住。）

4.4 一条工程纪律：判据的杀伤半径必须在启用之前量出来

修一个 bug 的规则本身常常会制造更多错误。本项目把这条做成流程：
任何"改期间/改列结构"的新规则，启用前先用量测工具在真实语料上量出 命中率与附带损害。

这条纪律有直接的代价记录：有一轮守卫规则"写完再量"，把 needs_review 从 159 条推到 529
条、可用净资产从 16 条掉到 0 条，只能整条回退； 另一轮先写量测工具，在 987
张真实表上两次否掉自己的判据， 第三次才收敛到 4/987 张、零误报。

五、验证与实测数据

5.1 验证方法：十八轮真实数据验证

全部结论建立在真实公开材料上，不使用合成数据作为证据：

语料	规模	来源
真实年报	4 份 / 943 页（中国石油 2024、2025；湖南发展 2025；湖南白银 2025）	巨潮资讯网
真实公告	77 份（质押 13 / 中标 12 / 股权变动 19 / 利润分配 11 / 对外担保 22，共 923 页）	巨潮资讯网
其中无文本层扫描件	106 页 / 14 份（已渲染页面人工核对，确认均为纯文字扫描件）	同上

> 说明（一）：这 77 份公告已于 2026-09-27 跑通，产物在 runs/ann77-20260927（7/7 阶段，765 条事实，47/77 份文件有命中，即 36/62 份不同文档）。
> 赛题点名的三类字段都有实测条数：质押 31 / 中标 30 / 股权变动 46 = 107 条（毛数）。但 107 是下界：视觉通道 400 次预算用尽（84 次整页扫描 + 316 格裁决），闸门用尽后 judge.py 静默 break 掉后面所有表格的裁决，且不留任何记录；另有 382 条候选被「单位缺失」闸门否决。材料里到底有多少个该抽的字段没有金标准，需人工逐页点数才能给出分母。
>
> 说明（二）：「77 份」里有 15 对是同一份文档。 doc_id 按内容哈希生成，语料里 15 对文件的 sha256 完全相同（同一份公告重复抓取，只有文件名里的 h 序号不同）—— 所以 77 份文件只对应 62 份不同文档（923 页对应 681 个不同页）。任何按 doc_id 汇总的统计都会把这 15 对重复计一次：我们在演示台 /library 上就是因为这个出现了「卡片 158 / 明细列 270」同页矛盾，已修（明细列改数每份文件自己的路由，重复行加「重复 ×N」标记）。这批重复文件我们没有删 —— ingest 去重属于 pipelines/ 的计算逻辑，按约定不动；且删除语料会改变已冻结 run 的输入指纹。是否清理由你决定。
>
> 说明（三）：此处曾写「296 份」，那是把当时 agent/sample/_scratch/ 下 219 份无分类标签的抓取临时件一并算进来的历史规模。那些临时件不在本仓、也从不随源码包提交（打包器按 _ 前缀规则排除 _ 开头的目录），所以今天在仓库里跑 --data agent/sample 与在交付包里跑，ingest 的都是 77 份这一档，输入集一致；runs/ann77-20260927 记录的 data_dir 正是 agent/sample，在仓库里可以直接按它复现。

每一轮验证都记录：假设 → 命令 → 实测输出 → 结论 → 被推翻的中间结论， 共十八轮，全部留档在 docs/VALIDATION_ROUND*.md，索引见 docs/VALIDATION_INDEX.md。

5.2 关键指标（含出处）

指标	值	出处
公告语料规模	765 条事实 / 923 页 / 77 份文件（= 62 份不同文档；47/77 份文件、36/62 份文档有命中）	runs/ann77-20260927
公告抽取路径构成	确定性 68.0% / 模型 32.0%（rule_engine 292、table_grid 228、vlm_scan 245）	同上
赛题点名三类字段命中	质押 31 / 中标 30 / 股权变动 46（毛数，下界）	同上
视觉通道预算	400 / 400 用尽（84 次整页扫描 + 316 格裁决）→ 后续表格静默跳过	同上
事实库规模	999 条事实 / 943 页 / 4 份年报	runs/ab-no4
抽取路径构成	确定性 80.3% / 模型 19.7%（table_grid 776、rule_engine 26、vlm_scan 197）	同上
模型读数与确定性路径冲突	0 条	同上
表格列结构按坐标重建	730 / 987 张	tools/grid_align_report.py
表格 T1 保真 / 编造	100% / 0	55 张真实表格
OCR 字符准确率 / 数字召回	95–98% / 89–93%	受控退化实验（docs/P0_VALIDATION.md）
R4 相对估值区间	¥12.1554 ~ ¥13.1668（6 家真实同业）	runs/web-20260927-123902
R4 DCF 点估计	¥14.0394	同上
R4 折现率	WACC 6.22%（CAPM，Rf 来自中债日评缓存）	data/market/rates_2026-09-17.json
R5 纠错核查	P 1.0000 / R 0.5000 / F1 0.6667（零误报）	runs/r5-eval-annual16
测试套件	736 passed, 2 skipped	python -m pytest -q（可当场复现）
单份材料成本	185 页年报 约 18 分钟 / ¥0.96	README 实测

口径说明（重要）：上表中「可当场复现」的只有测试套件与治理面；
其余指标出自真实模型运行。runs/ 不入版本库，
因此这些运行在全新克隆上需要按 docs/REVIEWER.md 第五节重建
（四份年报合计约 1 小时 / 数元；77 份公告约 28 分钟 / ¥2.02，
400 次模型调用、缓存命中 52.0%）。我们不把"需要重建"包装成"随时可复现"。

公告那一档的成本口径有一处不一致，写明：audit.md 记 ¥2.0222
（输入 1,018,093 / 输出 90,845 token），而 checkpoint.json 的
cost_cny 记 ¥1.0922（token 563,085）。后者恰为前者的 50.8%，
与「缓存命中 208/400 = 52.0%」接近 —— 看起来是不计缓存命中那部分的口径，
但根因未逐行核实，所以这里只写现象与推测。引用时以 audit.md 为准。
这是已知的、未修的数字不一致。

表里的事实库规模与抽取路径构成取自 runs/ab-no4，它是本机现存、
能用 python -m agent.cli replay runs/ab-no4 --strict 复算的整语料真实运行。
早先文档写的 1050 条 / 确定性 87.7% 出自 runs/annual41-seg，
该运行的产物不在本机，这两个数字在本次提交里无法复算；
同一语料上实测得到的是 999 条 / 80.3%。

5.3 缺陷与诚实披露

· 十八轮共修掉 34 处缺陷，其中 12 处属于静默错误（数字看着对、实际错），
例如：行标签单位被漏看导致现金放大 100 倍；表头与数据按列号强行配对， 拼出 '12目'
这类标签而"数字对、标签全错"。

· 留档 4 处被实测否决的改动（含一处把 needs_review 从 159 推到 529 的守卫）——
写进来是为了说明"我们量过、也退过"，不是只展示成功。

· 明确记录 8 条未解决问题（含根因与量化影响），例如
「阶段间仍有内存态」这条结构性缺陷，以及若干跨页归属错标。 它们写进 README
的「已知限制」，也写进 docs/VALIDATION_INDEX.md 第五节。

六、工程实现与可复现性

6.1 一键复现（评审入口）

```
python tools/reproduce_all.py
```

实测输出（2026-09-25 / Python 3.13.14 / Windows）：

```
□ 环境自检（依赖 / 配置 / 数据目录）（2.3s）
□ 数据契约导出（6 个 JSON Schema）（0.8s）
□ 技能注册表：绑定校验（声明的工具/提示词真实存在）（0.9s）
□ 提示词注册表（版本 + SHA256 指纹）（0.7s）
□ 工具注册表（副作用 / 确定性声明）（0.9s）
□ MCP 客户端自检（真实子进程 + 完整协议往返）（1.1s）
□ 端到端跑通 R1 → R4 → R5（mock 模型，零成本）（4.8s）
□ 产物完整性（34 个产物的哈希与 manifest 一致）（0.7s）
□ 离线回放并逐字节比对（不调用任何模型）（6.1s）
□ 生成人类可读审计报告（过程可追溯）（0.8s）
□ Web 演示台渲染核验（进程内，不占端口）（4.5s）
□ 测试套件（全量）（92.2s）

汇总：□ 全部通过（12/12）
```

每一步都有判据，不是"返回码为 0 就算过"：可复现性必须真的看到 □ 可复现 字样 —— 因为「退出码 0 但结论是不可复现」是最坏的情况。

6.2 测试

736 个测试（736 passed, 2 skipped），覆盖单元、集成、端到端与文档诚信四类，其中包含若干"防漂移"体检：文档里声称的模块必须真实存在、独立脚本必须切 UTF-8 控制台编码、类体被切开必须被抓到。这些体检都带注入验证（往文档里塞一个不存在的模块名，检查必须报错）—— 一条永远通过的检查等于没有检查。

6.3 交付物

类别	位置
作品说明（总入口）	README.md
评审复现指引	docs/REVIEWER.md
赛题对照与差距清单	docs/COMPETITION_FIT.md
技术验证总览（十八轮）	docs/VALIDATION_INDEX.md + docs/VALIDATION_ROUND*.md
第三方依赖声明	THIRD_PARTY.md

类别	位置
数据契约	configs/schemas/*.json
现场演示	tools/demo_run.py + demo_materials/
本计划书	deliverables/项目计划书.md（PDF 由 tools/make_plan_pdf.py 生成）

6.4 第三方依赖与边界（赛题技术要求第 7、8 条）

第三方模型的名称、版本、调用方式、使用范围，以及开源项目的
名称、版本、来源、许可证、使用范围，逐条列在 THIRD_PARTY.md； 其中包含
PyMuPDF（AGPL-3.0）风险的主动披露。 核心编排、工具、提示词、技能、MCP
客户端、追踪与复现机制均自研， 不使用第三方编排框架。

七、应用价值

1. 可直接嵌入券商/基金的研究流程：把"人工翻年报、逐页核对"变成 "机器抽事实库 + 人工只审复核队列"，且每条数字带原文锚点，复核成本大幅下降。

2. 可迁移到合规与审计场景：报告纠错核查（R5）的五类规则与误报控制机制，天然适配"研究报告质控 / 披露文件校对"。

3. 方法论价值：本项目最有价值的产出不是某一条规则，而是一套让错误无法静默 通过的做法 —— 无锚点不入库、模型改写必须有几何事实背书、 判据启用前先量杀伤半径、文档也必须被测试。

4. 成本可控：单份 185 页年报约 ¥0.96，且"赛前预热 + 现场续跑"可把 现场演示压到秒级。

八、团队分工与进度

8.1 分工（3–5 人）

角色	负责范围	对应赛题
队长 / 架构	Agent Harness（编排、预算、追踪、复现）、数据契约	技术要求 4/5/6
抽取方向	三层解析引擎、表格网格重建、单元格裁决、OCR	选题 1
建模方向	估值模型、假设推导、勾稽断言、敏感性	选题 4
核查方向	纠错规则、误报控制、评测集与指标	选题 5
演示 / 交付	Web 演示台、现场流程、文档与视频	现场展示与答辩

> 待填写：队员姓名、学校、年级、指导老师（晋级决赛后须配 1 名指导老师）。

8.2 进度

阶段	时间	交付物	状态
赛题研读与选型	9/10 – 9/15	选题确定（1/4/5）	完成
核心链路打通	9/15 – 9/18	R1→R4→R5 端到端跑通	完成
真实材料验证	9/18 – 9/22	十八轮验证与缺陷修复	完成
初赛交付	9/23 – 10/18	计划书 PDF + ≤5 分钟视频	进行中
决赛打磨	10/18 – 11 月底	最终报告书、源码包、现场演示与答辩	计划中

> 赛程硬节点：报名 9/10–10/11 → 作品提交截止 10/18 →
> 初赛评审（预计 11 月中旬公布入围）→ 11 月底决赛（现场展示 + 答辩）。

九、风险与合规

风险	应对
模型读数不稳定（同一材料两次冷跑结果不同）	赛前预热 + 现场续跑；现场要看重新解析则用 Web /process 的限量预览
材料版权与合规	只使用公开披露材料；真实材料不入版本库；受控数据环境默认禁止出网
估值被误读为投资建议	报告 applicability 字段显式声明适用范围；假设区分事实/推论/观点；"仅作示意"横幅置于结论之上
第三方许可证风险	THIRD_PARTY.md 逐条声明，含 AGPL 依赖的主动披露
现场演示失败	全流程有"零依赖 mock 路径"兜底；reproduce_all.py 可当场自证

十、已知限制

1. 阶段间仍有内存态（未根治）：跳过某些阶段续跑会丢失只存在于内存的状态；
已加"从产物水化"与"空库硬失败"两道保险，但结构性修法未做。

2. 报表归属跨页错标：整页高的续表会让标题落在表内；
试过按文档顺序修，被实测否决（口径与期间互相纠缠）。

3. 多年摘要表的期间仍可能错位：值可用、期间需人工核对。

4. 3 张分部表未对齐：系统返回"不猜"，那 3 张表的分部合计取不到。

5. EV/EBITDA 缺失：现金与有息负债没有可靠的结构化公开来源，相对估值少一个倍数。

6. 抽取结果对单格模型读数敏感：冷启动结果不完全可预期。

7. 续跑重跑阶段会留孤儿产物：操作规避是换新 run 名重跑。

8. DCF 与相对估值仍有约 7% 差距，来源已量化 （CAPM 算出的 WACC
低于市场隐含要求回报率）， 且不用市价反向校准 WACC —— 那会让 DCF
退化成把市价重算一遍。

> 完整的 8 条未解决问题、每条根因与量化影响见 docs/VALIDATION_INDEX.md 第五节
> 与 README.md 的「已知限制」。